Figure C.4 Inductive definition of a list. The middle column uses a formal notation for the list, while the right column shows the graphical representation of the same list. In the middle representation the recursive nature is more explicit, because it is indeed a recursive notation: the empty list is assumed as a primitive, a building block, and each non-empty list is recursively defined as its first element, followed by its tail, a shorter, possibly empty list of the remaining elements.

Figure C.5 exemplifies another key feature of lists, that can be either:

- *Singly linked*: every node has only a link (aka pointer or reference) to the next element.
- *Doubly linked*: two links per node are stored: a link to the next element and one to the previous element.

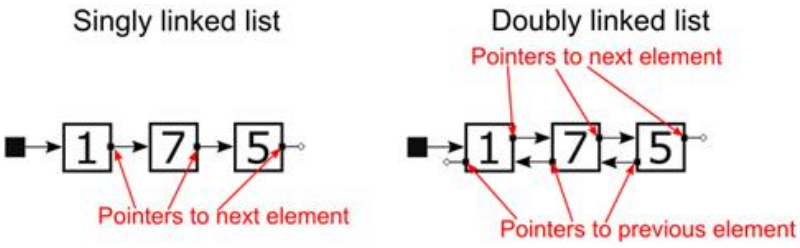


Figure C.5 Singly linked (left) versus doubly linked lists (right).

As mentioned before, the choice between singly and doubly linked is a trade-off: the former requires less space per node, the latter allows for faster algorithms to delete an element but needs a small overhead to keep the pointers updated.

For keeping track of a linked list, the head of the list is stored in a variable. When adding or removing elements from the list we need to take extra care to update that reference to the head of the list as well. If we don’t perform the update, it might point to an internal node or even worse to an invalid one.

Insertion in lists can happen:

- **At the beginning.**

  That's the easiest place to add a node, in constant time. Figure C.6 shows, in fact, that we only need update the head reference with a